RevoBank

Customer Segmentation & Transaction Analysis

By : Eva Arliana

Google Colab

Business Background

RevoBank is a digital bank in Indonesia that generates revenue primarily from credit card transactions. Each transaction contributes a 1.5% margin through Merchant Discount Rate (MDR) fees.

The Performance Management (PM) team aims to increase transaction activity from existing customers to drive higher revenue. To support this, RevoBank provides customer and card data from the past 6 months for analysis.

Since one customer can own multiple cards, understanding customer behavior requires combining both user and card data to capture a complete view of transaction patterns and financial profiles.

Goals & Disclaimer

Goal of Analysist

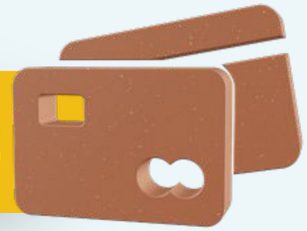
To analyze customer transaction behavior and financial characteristics in order to identify distinct customer segments that can help RevoBank increase transaction activity and maximize net profit.

Disclaimer

This analysis is conducted using a simulated dataset provided for learning purposes and does not represent actual RevoBank customer data.

Card Dataset

Data Cleaning - Card Dataset



Data Type Convertion

Conversion Summary

- id → from int to string
- client_id → from int to string
- card_number → from float to string
- expires → from object to datetime
- acct_open_date → from object to datetime
- credit_limit → from object to float
- total_transactions → from float to int
- total_amount → from string to float

Before

```
1 #display the structure and summary information of the dataframe
2 df_card_dc.info()
```

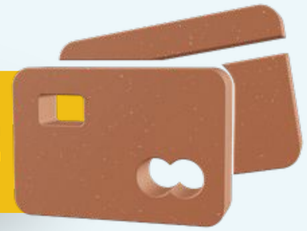
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5599 entries, 0 to 5598
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      5599 non-null   int64
 1   client_id               5599 non-null   int64
 2   card_brand              5599 non-null   object
 3   card_number             5599 non-null   float64
 4   expires                 5599 non-null   object
 5   cvv                     5599 non-null   int64
 6   credit_limit            5587 non-null   object
 7   acct_open_date          5599 non-null   object
 8   year_pin_last_changed   5599 non-null   int64
 9   days_since_last_trx     5599 non-null   int64
 10  count_nonfraud_trx_L6M  3707 non-null   float64
 11  amt_nonfraud_trx_L6M    3707 non-null   object
 12  count_fraud_trx_L6M     547 non-null    float64
 13  amt_fraud_trx_L6M       547 non-null    object
dtypes: float64(3), int64(5), object(6)
memory usage: 612.5+ KB
```

After

```
1 df_card_dc.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5599 entries, 0 to 5598
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      5599 non-null   object
 1   client_id               5599 non-null   object
 2   card_brand              5599 non-null   object
 3   card_number             5599 non-null   object
 4   expires                 5599 non-null   datetime64[ns]
 5   cvv                     5599 non-null   int64
 6   credit_limit            5587 non-null   float64
 7   acct_open_date          5599 non-null   datetime64[ns]
 8   year_pin_last_changed   5599 non-null   int64
 9   days_since_last_trx     5599 non-null   int64
 10  count_nonfraud_trx_L6M  5599 non-null   int64
 11  amt_nonfraud_trx_L6M    3707 non-null   float64
 12  count_fraud_trx_L6M     5599 non-null   int64
 13  amt_fraud_trx_L6M       547 non-null    float64
dtypes: datetime64[ns](2), float64(3), int64(5), object(4)
memory usage: 612.5+ KB
```

Data Cleaning - Card Dataset



Cek Values & Typos

Insight :

- The inconsistency in card_brand has been resolved ("Jcb" successfully standardized to "JCB").

Recommendation :

- Apply consistent standardization rules (e.g., case formatting) across all categorical columns.

```
1 #Checking values of card_brand
2 df_card_dc['card_brand'].value_counts()
```

Before

card_brand	count
Mastercard	2826
Visa	2162
Amex	402
JCB	206
Jcb	3

dtype: int64

```
1 df_card_dc['card_brand'].value_counts()
```

After

card_brand	count
Mastercard	2826
Visa	2162
Amex	402
JCB	209

dtype: int64

Cek Duplicate - Card Dataset

Before

```
1 #check the duplicate
2 df_card_dc.duplicated().sum()
```

```
np.int64(31)
```

```
1 #check which rows are duplicated based on a specific column
2 df_card_dc[df_card_dc.duplicated(subset=['id'], keep=False)].head()
```

	id	client_id	card_brand	card_number	expires	cvv	credit_limit	acct_open_date	year_pin_last_changed	days_since_last_trx
20	20	1728	Visa	4665365862082531	2027-06-01	412	5962000.0	2006-01-01	2008	25
23	24	1274	Visa	4181524038316536	2027-04-01	2	23786000.0	2006-01-01	2012	4
191	209	1334	Visa	4144931322114018	2029-02-01	312	14592000.0	2014-01-01	2014	32
340	374	1115	Visa	4906410000000000	2029-02-01	579	15566000.0	2023-01-01	2023	36
667	749	54	Visa	4557061225558443	2029-05-01	780	22300000.0	2023-05-01	2025	604

After

```
1 #apply to the original dataframe
2 df_card_dc = df_card_dc.drop_duplicates(subset=['id'], keep='last')
```

```
1 df_card_dc['id'].duplicated().sum()
```

```
np.int64(0)
```

```
1 df_card_dc.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5568 entries, 0 to 5598
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      5568 non-null   object
 1   client_id               5568 non-null   object
 2   card_brand              5568 non-null   object
 3   card_number             5568 non-null   object
 4   expires                 5568 non-null   datetime64[ns]
 5   cvv                     5568 non-null   int64
 6   credit_limit            5568 non-null   float64
 7   acct_open_date          5568 non-null   datetime64[ns]
 8   year_pin_last_changed   5568 non-null   int64
 9   days_since_last_trx     5568 non-null   int64
 10  count_nonfraud_trx_L6M  5568 non-null   int64
 11  amt_nonfraud_trx_L6M    5568 non-null   float64
 12  count_fraud_trx_L6M     5568 non-null   int64
 13  amt_fraud_trx_L6M       5568 non-null   float64
dtypes: datetime64[ns](2), float64(3), int64(5), object(4)
memory usage: 652.5+ KB
```

Insight :

- Duplicate rows were identified where all column values are identical.
- These duplicates may lead to double counting in analysis.

Recommendation :

- Remove duplicate rows to ensure data accuracy and prevent redundant records.

Data Cleaning - Card Dataset

Remove Expired Card

Insight :

- Expiry dates were correctly standardized to end-of-month, ensuring accurate business interpretation of card validity.
- 37 records were removed as they were already expired or had zero credit limit, resulting in a cleaner and more relevant dataset for analysis.

Recommendation :

- Enforce consistent date formatting (end-of-month for expiry) during data ingestion to avoid misclassification.
- Apply business rule validation (exclude expired or inactive cards) as a standard preprocessing step for future datasets.

Before

```
df_card_dc.shape
```

```
(5568, 14)
```

```
#preview transformation
temp = df_card_dc.copy()

temp['expires_fixed'] = (
    pd.to_datetime(temp['expires'], format='%m/%Y')
    + pd.offsets.MonthEnd(0)
)

temp[['expires', 'expires_fixed']].head()
```

	expires	expires_fixed
0	2030-04-01	2030-04-30
1	2030-06-01	2030-06-30
2	2027-09-01	2027-09-30
3	2026-04-01	2026-04-30
4	2030-03-01	2030-03-31

After

```
#apply transformation
df_card_dc['expires'] = (
    pd.to_datetime(df_card_dc['expires'], format='%m/%Y')
    + pd.offsets.MonthEnd(0)
)

df_card_dc = df_card_dc[
    (df_card_dc['expires'] >= cutoff_date) &
    (df_card_dc['credit_limit'] > 0)
]
```

```
df_card_dc.shape
```

```
(5531, 14)
```

Card Data Cleaning

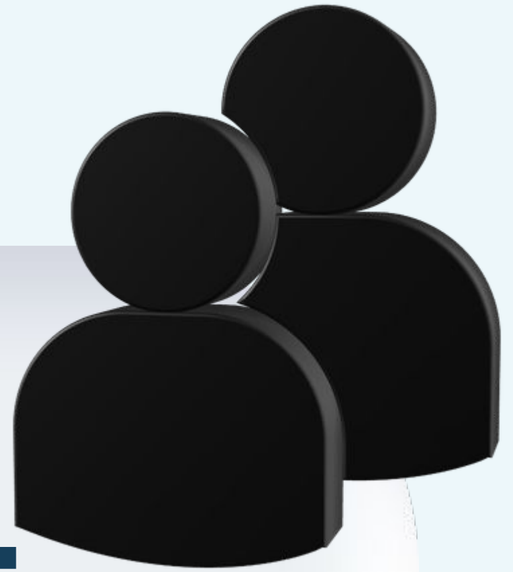
```
1 # create a copy of the dataframe for further processing without modifying the original
2 df_cards = df_card_dc.copy()
```

```
1 df_cards.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5531 entries, 0 to 5598
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      5531 non-null   object
 1   client_id               5531 non-null   object
 2   card_brand              5531 non-null   object
 3   card_number             5531 non-null   object
 4   expires                 5531 non-null   datetime64[ns]
 5   cvv                     5531 non-null   int64
 6   credit_limit            5531 non-null   float64
 7   acct_open_date          5531 non-null   datetime64[ns]
 8   year_pin_last_changed   5531 non-null   int64
 9   days_since_last_trx     5531 non-null   int64
 10  count_nonfraud_trx_L6M  5531 non-null   int64
 11  amt_nonfraud_trx_L6M    5531 non-null   float64
 12  count_fraud_trx_L6M     5531 non-null   int64
 13  amt_fraud_trx_L6M       5531 non-null   float64
dtypes: datetime64[ns](2), float64(3), int64(5), object(4)
memory usage: 648.2+ KB
```

A final cleaned dataset has been created to ensure the data is ready and structured for further analysis.

User Dataset



Data Cleaning - User Dataset

Data Type Convertion

Conversion Summary

- id → from int to string
- birthdate →from string to datetime
- per_capita_income →from string to float
- yearly_income → from string to float
- total_debt → from string to float

Before

```
1 df_user_dc.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   id                  2000 non-null   int64
 1   retirement_age      2000 non-null   int64
 2   birthdate           2000 non-null   object
 3   gender              2000 non-null   object
 4   per_capita_income   2000 non-null   object
 5   yearly_income       2000 non-null   object
 6   total_debt          2000 non-null   object
 7   credit_score        2000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 125.1+ KB
```

After

```
1 df_user_dc.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   id                  2000 non-null   object
 1   retirement_age      2000 non-null   int64
 2   birthdate           2000 non-null   datetime64[ns]
 3   gender              2000 non-null   object
 4   per_capita_income   2000 non-null   int64
 5   yearly_income       2000 non-null   int64
 6   total_debt          2000 non-null   int64
 7   credit_score        2000 non-null   int64
dtypes: datetime64[ns](1), int64(5), object(2)
memory usage: 125.1+ KB
```

Data Cleaning - User Dataset

Cek Values & Typos

Insight :

- The columns per_capita_income, yearly_income, and total_debt are stored as object due to the presence of "Rp" and dot separators.
- The data uses Indonesian number format (dot as thousand separator).
- Direct conversion without proper handling may lead to many missing values (NaN).

Recommendation :

- Remove non-numeric characters such as "Rp" and thousand separators.
- Apply appropriate cleaning based on Indonesian number format.
- Convert the columns into numeric (float) after cleaning to ensure accurate analysis.

Before

```
df_user_dc[['per_capita_income', 'yearly_income', 'total_debt']].head()
```

	per_capita_income	yearly_income	total_debt
0	Rp45.937.000	Rp93.663.000	Rp38.138.095
1	Rp59.451.000	Rp121.212.000	Rp57.186.095
2	Rp35.586.000	Rp52.535.000	Rp58.666
3	Rp255.975.000	Rp392.132.000	Rp60.467.238
4	Rp84.407.000	Rp172.099.000	Rp54.946.285

After

```
#preview cleaned numeric values by converting to string, removing non-numeric characters
#(except commas and minus signs), and converting to numeric (invalid parsing set to NaN)
pd.DataFrame({
    col: pd.to_numeric(
        df_user_dc[col].astype(str)
        .replace('[^0-9,-]', '', regex=True),  # remove dot
        errors='coerce'
    )
    for col in ['per_capita_income', 'yearly_income', 'total_debt']
}).head()
```

	per_capita_income	yearly_income	total_debt
0	45937000	93663000	38138095
1	59451000	121212000	57186095
2	35586000	52535000	58666
3	255975000	392132000	60467238
4	84407000	172099000	54946285

Data Cleaning - User Dataset

Missing & Irrelevant Value

Insight :

- The user dataset contains no missing values across all columns.
- No irrelevant or unrealistic values were identified in the user dataset.
- This indicates high data completeness and good data quality.

Recommendation :

- No missing value treatment is required.
- No data removal or filtering is required, as all values are considered valid.
- The dataset can be directly used for further analysis.

```
1 df_user_dc.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   id                  2000 non-null   object
 1   retirement_age      2000 non-null   int64
 2   birthdate           2000 non-null   datetime64[ns]
 3   gender              2000 non-null   object
 4   per_capita_income   2000 non-null   int64
 5   yearly_income       2000 non-null   int64
 6   total_debt          2000 non-null   int64
 7   credit_score        2000 non-null   int64
dtypes: datetime64[ns](1), int64(5), object(2)
memory usage: 125.1+ KB
```

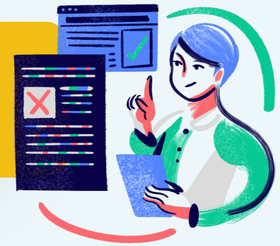
No Missing Value

```
1 #generate summary statistics for the yearly_income and total_debt column
2 df_user_dc[['yearly_income', 'total_debt']].describe()
```

	yearly_income	total_debt
count	2.000000e+03	2.000000e+03
mean	7.172821e+07	1.904010e+07
std	3.607541e+07	1.561662e+07
min	2.000000e+03	0.000000e+00
25%	5.149225e+07	7.168524e+06
50%	6.392800e+07	1.740867e+07
75%	8.268400e+07	2.661938e+07
max	4.817110e+08	1.542890e+08

No Irrelevant Value

Data Manipulation - User Dataset



Age Column

```
reference_date = pd.to_datetime('2025-05-31')

df_user_dc['age'] = (
    (reference_date - df_user_dc['birthdate']).dt.days // 365.25
).astype(int)
```

```
df_user_dc[['birthdate', 'age']].head()
```

	birthdate	age
0	1972-11-25	52
1	1972-12-16	52
2	1944-11-04	80
3	1963-01-12	62
4	1982-09-21	42

DTI Column

```
df_user_dc['DTI'] = np.where(
    df_user_dc['yearly_income'] > 0,
    df_user_dc['total_debt'] / df_user_dc['yearly_income'],
    np.nan
)
```

```
df_user_dc[['total_debt', 'yearly_income', 'DTI']].head()
```

	total_debt	yearly_income	DTI
0	38138095	93663000	0.407184
1	57186095	121212000	0.471786
2	58666	52535000	0.001117
3	60467238	392132000	0.154201
4	54946285	172099000	0.319271

retired_flag Column

```
#create a binary retirement flag using np.where based on age and retirement_age
df_user_dc['retired_flag'] = np.where(
    df_user_dc['age'] >= df_user_dc['retirement_age'],
    1,
    0
)
```

```
df_user_dc[['age', 'retirement_age', 'retired_flag']].head()
```

	age	retirement_age	retired_flag
0	52	66	0
1	52	68	0
2	80	67	1
3	62	63	0
4	42	70	0

Cek Duplicate - User Dataset

```
1 #check the duplicate
2 df_user_dc[df_user_dc['id'].duplicated()]
```

No duplicate

id	retirement_age	birthdate	gender	per_capita_income	yearly_income	total_debt	credit_score	age	retired_flag	DTI

```
1 #count the number of duplicate values in the id column
2 df_user_dc['id'].duplicated().sum()
```

```
np.int64(0)
```

Insight :

- No duplicate values were found in the user ID column, indicating that each user record is unique.

Recommendation :

- No duplicate removal is required as the dataset is already clean and reliable.

User Data Cleaning

```
1 #for summary after data cleaning
2 df_users.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   id                    2000 non-null   object
 1   retirement_age        2000 non-null   int64
 2   birthdate             2000 non-null   datetime64[ns]
 3   gender                2000 non-null   object
 4   per_capita_income     2000 non-null   int64
 5   yearly_income         2000 non-null   int64
 6   total_debt            2000 non-null   int64
 7   credit_score          2000 non-null   int64
 8   age                   2000 non-null   int64
 9   retired_flag          2000 non-null   int64
 10  DTI                   2000 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(7), object(2)
memory usage: 172.0+ KB
```

A final cleaned dataset has been created to ensure the data is ready and structured for further analysis.

Merge Data

Merge dataset use left join :

```
df_merge = df_cards.merge(
    df_users,
    left_on='client_id',
    right_on='id',
    how='left'
)
```

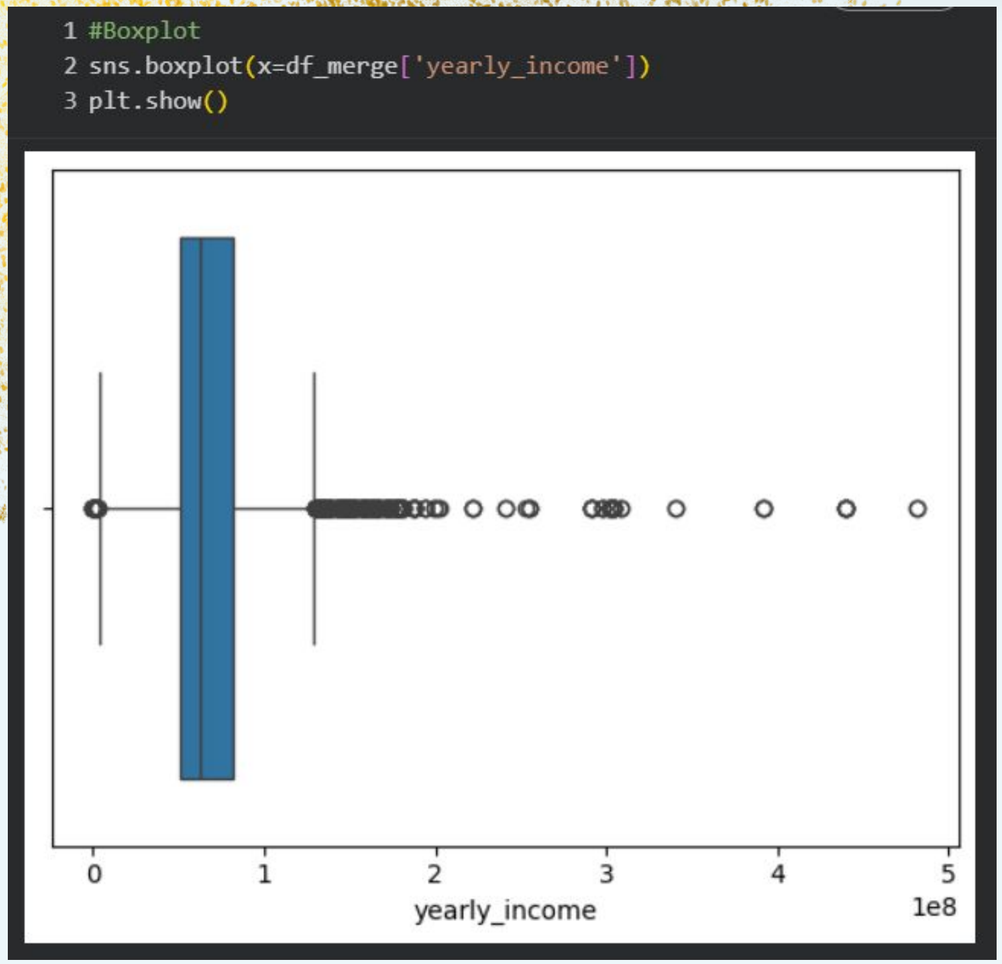
A left join was performed to combine card and user datasets while preserving all card records.

```
1 #to summary the data cleaning
2 df_merge.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5531 entries, 0 to 5530
Data columns (total 25 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id_x                    5531 non-null   object
 1   client_id               5531 non-null   object
 2   card_brand              5531 non-null   object
 3   card_number             5531 non-null   object
 4   expires                 5531 non-null   datetime64[ns]
 5   cvv                     5531 non-null   int64
 6   credit_limit            5531 non-null   float64
 7   acct_open_date          5531 non-null   datetime64[ns]
 8   year_pin_last_changed   5531 non-null   int64
 9   days_since_last_trx     5531 non-null   int64
 10  count_nonfraud_trx_L6M  5531 non-null   int64
 11  amt_nonfraud_trx_L6M    5531 non-null   float64
 12  count_fraud_trx_L6M     5531 non-null   int64
 13  amt_fraud_trx_L6M       5531 non-null   float64
 14  id_y                    5531 non-null   object
 15  retirement_age          5531 non-null   int64
 16  birthdate               5531 non-null   datetime64[ns]
 17  gender                  5531 non-null   object
 18  per_capita_income       5531 non-null   int64
 19  yearly_income           5531 non-null   int64
 20  total_debt              5531 non-null   int64
 21  credit_score            5531 non-null   int64
 22  age                     5531 non-null   int64
 23  retired_flag            5531 non-null   int64
 24  DTI                     5531 non-null   float64
dtypes: datetime64[ns](3), float64(4), int64(12), object(6)
memory usage: 1.1+ MB
```

Outlier

```
1 #Boxplot
2 sns.boxplot(x=df_merge['yearly_income'])
3 plt.show()
```



Insight :

- Outliers were identified in the yearly_income column, primarily on the higher end of the distribution.
- These values likely represent high-income users rather than data errors.

Recommendation :

- Outliers should not be removed as they may provide valuable insights into high-value customer segments.
- Consider analyzing these users separately for more targeted insights.

Calculate IQR

```
Q1 = df_merge['yearly_income'].quantile(0.25)
Q3 = df_merge['yearly_income'].quantile(0.75)
IQR = Q3 - Q1

LB = Q1 - 1.5 * IQR
UB = Q3 + 1.5 * IQR

print(Q1, Q3, IQR, LB, UB)
```

Insight :

- Outliers were identified using the IQR method in the yearly_income column.
- Most outliers are located above the upper bound, indicating high-income users.
- There are minimal or no outliers below the lower bound.

Recommendation :

- Outliers should not be removed as they represent valid high-income customers.
- Consider segmenting these users for further analysis to identify high-value opportunities.

```
#checking outlier without remove
df_merge[(df_merge['yearly_income'] < LB) | (df_merge['yearly_income'] > UB)]
```

	id_x	client_id	card_brand	card_number	expires	cvv	credit_limit	acct_open_date	year_pin_last_changed	days_since_last_trx
12	12	1156	Visa	4008696033204121	2029-02-28	124	89747000.0	2004-01-01	2016	11
16	16	1201	Visa	4817273521248161	2029-10-31	706	87382000.0	2005-01-01	2011	1
19	19	115	Mastercard	5610743457688598	2029-01-31	310	72463000.0	2006-01-01	2019	6
33	37	840	Mastercard	5486901761554156	2029-10-31	258	8267000.0	2008-01-01	2021	10
34	38	115	Mastercard	5419421476993334	2030-11-30	290	31851000.0	2008-01-01	2020	16

Net Profit Analysis (Last 6 Months)

Insight :

- The bank generated a positive net profit from transaction activities in the last 6 months.
- Non-fraud transactions significantly outweigh fraud losses, indicating healthy transaction performance.

Recommendation :

- Continue increasing transaction volume through promotions and user engagement.
- Maintain fraud monitoring to protect profit margins.

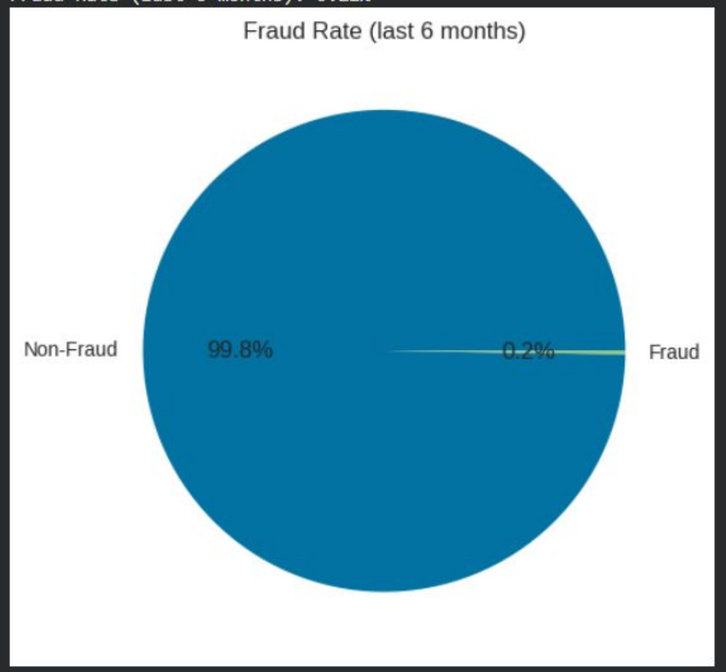
```python
#step 1 : calculate total non-fraud and fraud transaction amounts (last 6 months)
sum_nonfraud_amt = df_cards['amt_nonfraud_trx_L6M'].sum()
sum_fraud_amt = df_cards['amt_fraud_trx_L6M'].sum()

#step 2 : compute net profit using MDR rate (1.5%)
#net Profit = (MDR * total non-fraud amount) - total fraud amount
net_profit = 0.015 * sum_nonfraud_amt - sum_fraud_amt

#step 3 : display the result
print(f"Net Profit (last 6 months): Rp{net_profit:,.0f}")
```

```
Net Profit (last 6 months): Rp5,827,469,179
```

Fraud Rate Analysis (Last 6 Months)

```python
#Fraud Rate = SUM(Fraud Amount) / (SUM(Non-Fraud Amount) + SUM(Fraud Amount))

#step 1 : calculate Fraud Rate
fraud_rate = sum_fraud_amt / (sum_nonfraud_amt + sum_fraud_amt)

#step 2 : Display result
print(f"Fraud Rate (last 6 months): {fraud_rate:.2%}")

#step 3 : Create pie chart
plt.pie([sum_nonfraud_amt, sum_fraud_amt],
        labels=['Non-Fraud', 'Fraud'],
        autopct='%1.1f%%')

plt.title('Fraud Rate (last 6 months)')
plt.show()
```

```
Fraud Rate (last 6 months): 0.22%
```



Insight :

- Fraud rate is extremely low (~0.22%), indicating strong fraud control.
- The majority of transactions are non-fraud, showing stable operational performance.

Recommendation :

- Maintain and enhance fraud detection systems
- Focus more on scaling transaction volume rather than over-optimizing fraud controls.

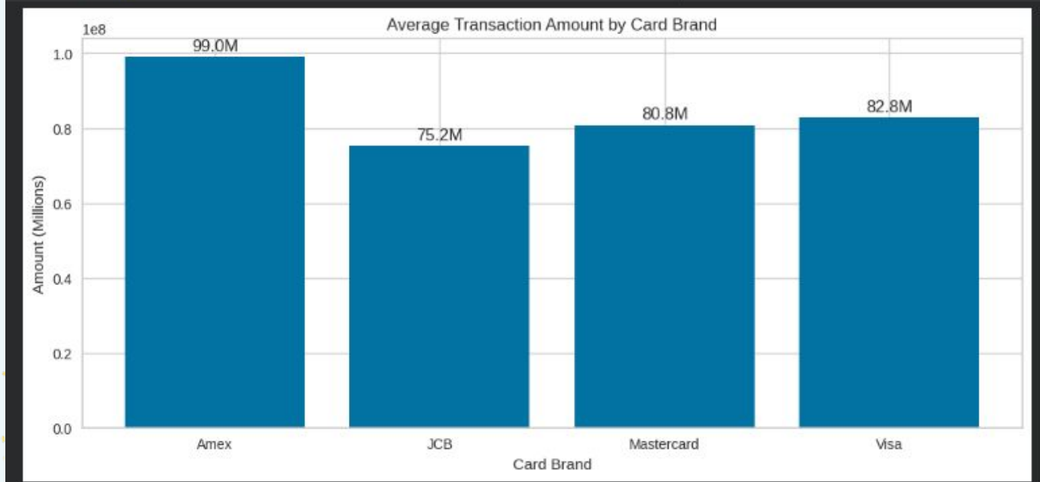
Transaction Behavior Analysis by Card Brand

```python
#card brand activity pattern (amount)
card_brand_amount = df_cards.groupby(['card_brand'])[['amt_nonfraud_trx_L6M', 'amt_fraud_trx_L6M']].describe().T

with pd.option_context('display.float_format', '{:,.2f}'.format):
    display(card_brand_amount)
```

	card_brand	Amex	JCB	Mastercard	Visa
amt_nonfraud_trx_L6M	count	395.00	205.00	2,813.00	2,118.00
	mean	98,830,780.76	75,026,117.07	80,595,841.17	82,588,236.54
	std	154,185,879.62	104,422,459.45	114,388,018.14	116,845,563.19
	min	0.00	0.00	0.00	-8,462,200.00
	25%	0.00	0.00	0.00	0.00
	50%	52,061,000.00	41,883,100.00	41,841,600.00	44,704,200.00
	75%	134,099,500.00	102,883,400.00	116,677,100.00	116,053,075.00
	max	1,553,045,500.00	800,335,300.00	1,231,907,200.00	1,071,909,000.00
amt_fraud_trx_L6M	count	395.00	205.00	2,813.00	2,118.00
	mean	206,937.72	185,500.00	191,013.58	168,219.64
	std	1,147,563.67	859,196.20	1,105,696.56	1,031,477.35
	min	-6,511,400.00	0.00	-5,177,700.00	-7,421,400.00
	25%	0.00	0.00	0.00	0.00
	50%	0.00	0.00	0.00	0.00
	75%	0.00	0.00	0.00	0.00
	max	12,620,400.00	7,886,700.00	26,958,500.00	17,280,800.00

Amount

```python
#create mean_amount
mean_amount = df_cards.groupby('card_brand')[['amt_nonfraud_trx_L6M', 'amt_fraud_trx_L6M']].mean()

#step: calculate total
total_amount = mean_amount['amt_nonfraud_trx_L6M'] + mean_amount['amt_fraud_trx_L6M']

#plot
fig, ax = plt.subplots(figsize=(12,5))

bars = ax.bar(mean_amount.index, total_amount)

#add label (clean format)
ax.bar_label(bars, labels=[f'{x/1e6:.1f}M' for x in total_amount], padding=2)

#styling
ax.set_title('Average Transaction Amount by Card Brand')
ax.set_xlabel('Card Brand')
ax.set_ylabel('Amount (Millions)')
fig.show()
```



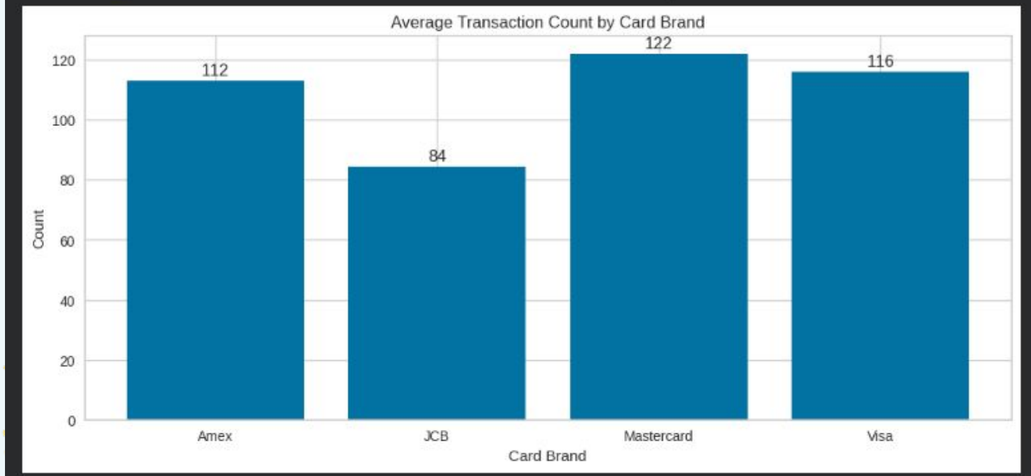
Transaction Behavior Analysis by Card Brand

```python
#card brand activity pattern (count)
card_brand_count = df_cards.groupby(['card_brand'])[['count_nonfraud_trx_L6M', 'count_fraud_trx_L6M']].describe().T

with pd.option_context('display.float_format', '{:,.0f}'.format):
    display(card_brand_count)
```

	card_brand	Amex	JCB	Mastercard	Visa
count_nonfraud_trx_L6M	count	395	205	2,813	2,118
	mean	113	84	122	116
	std	175	107	156	143
	min	0	0	0	0
	25%	0	0	0	0
	50%	66	58	80	76
	75%	151	118	185	181
	max	1,691	550	1,654	1,009
count_fraud_trx_L6M	count	395	205	2,813	2,118
	mean	0	0	0	0
	std	0	0	0	0
	min	0	0	0	0
	25%	0	0	0	0
	50%	0	0	0	0
	75%	0	0	0	0
	max	2	4	3	3

Count

```python
#step 1 : calculate mean count per card brand
mean_count = df_cards.groupby('card_brand')[['count_nonfraud_trx_L6M', 'count_fraud_trx_L6M']].mean()

# step 2 : calculate total count (biar chart lebih clean)
total_count = mean_count['count_nonfraud_trx_L6M'] + mean_count['count_fraud_trx_L6M']

# step 3 : plot
fig, ax = plt.subplots(figsize=(12,5))

bars = ax.bar(mean_count.index, total_count)

#step 4 : add label
ax.bar_label(bars, labels=[f'{int(x)}' for x in total_count], padding=2)

#step 5 : styling
ax.set_title('Average Transaction Count by Card Brand')
ax.set_xlabel('Card Brand')
ax.set_ylabel('Count')

fig.show()
```



Transaction Behavior Analysis by Card Brand

Insight

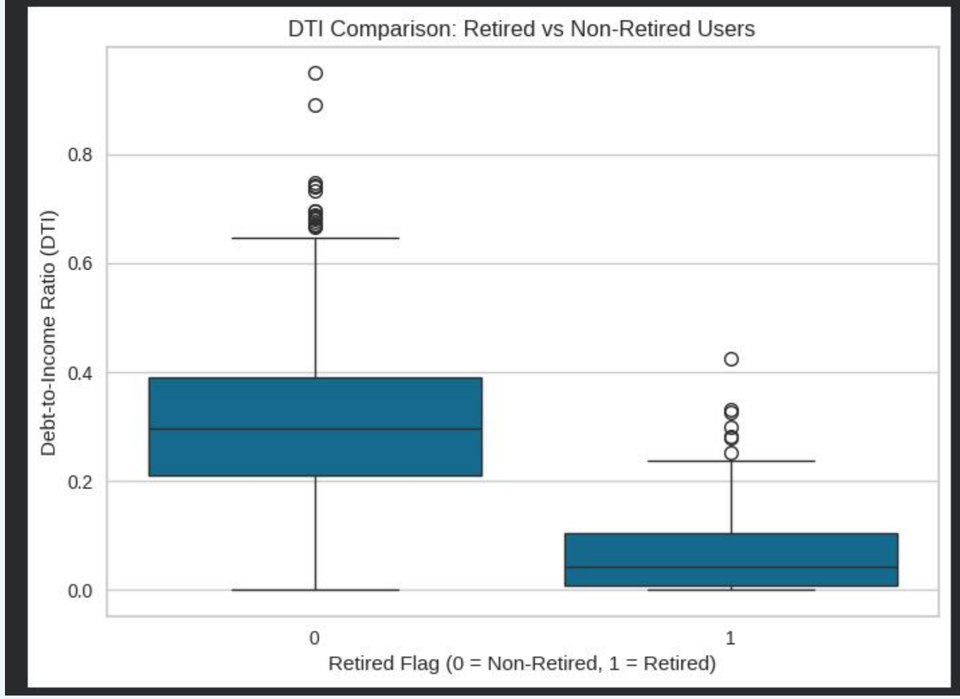
- Transaction activity varies across card brands in both amount and frequency.
- Some brands are high-frequency but lower value, while others generate higher transaction amounts.

Recommendation

- Focus marketing on high-value brands to maximize revenue (increase usage of cards with higher average transaction amount).
- Increase engagement for high-frequency brands through promotions or rewards.

DTI Analysis by Retirement Status

```
#visualize DTI comparison
sns.boxplot(data=df_users, x='retired_flag', y='DTI')

plt.title('DTI Comparison: Retired vs Non-Retired Users')
plt.xlabel('Retired Flag (0 = Non-Retired, 1 = Retired)')
plt.ylabel('Debt-to-Income Ratio (DTI)')
plt.show()
```



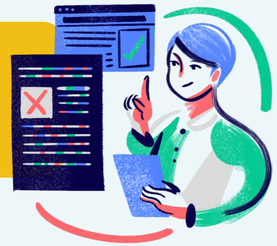
Insight :

- Non-retired users have significantly higher DTI, indicating higher financial risk.
- Retired users show lower and more stable DTI levels.
- High DTI outliers are mainly found among non-retired users.

Recommendation :

- Prioritize risk monitoring for non-retired users with high DTI.
- Apply stricter credit policies for users with elevated DTI.

Average Debt-to-Income (DTI) by Credit Score Category



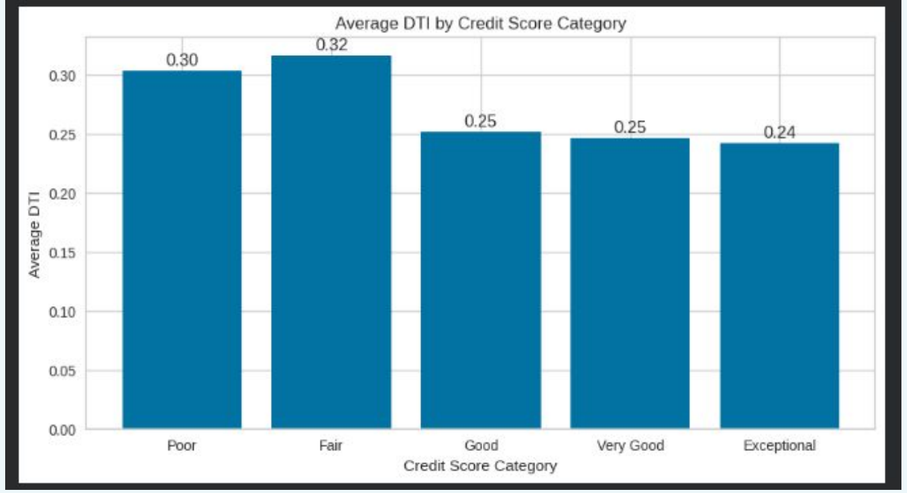
```python
#create credit score category
df_users['credit_score_category'] = pd.cut(
    df_users['credit_score'],
    bins=[300, 580, 670, 740, 800, 850],
    include_lowest=True,
    labels=['Poor', 'Fair', 'Good', 'Very Good', 'Exceptional']
)
```

```python
#hitung rata-rata DTI per credit_score_category
mean_dti_per_category = df_users.groupby(['credit_score_category']).agg({
    'DTI': 'mean'
})

#show
mean_dti_per_category
```

```
/tmp/ipykernel_13808/1306592966.py:2: FutureWarning: The default of observed=Fal
  mean_dti_per_category = df_users.groupby(['credit_score_category']).agg({
```

credit_score_category	DTI
Poor	0.303444
Fair	0.316493
Good	0.251562
Very Good	0.246766
Exceptional	0.242278

```python
#step 1 : define figure and axes
fig, ax = plt.subplots(figsize=(10,5))

#step 2 : draw bar chart
bars = ax.bar(mean_dti_per_category.index, mean_dti_per_category['DTI'])

#step 3 : add label (biar lebih readable)
ax.bar_label(bars, labels=[f'{x:.2f}' for x in mean_dti_per_category['DTI']], padding=2)

#step 4 : styling
ax.set_title('Average DTI by Credit Score Category')
ax.set_xlabel('Credit Score Category')
ax.set_ylabel('Average DTI')

#step 5 : show
fig.show()
```



Insight :

- DTI generally decreases as credit score improves, showing alignment between risk and creditworthiness.
- The Fair category has the highest DTI, indicating a potential risk concentration.

Recommendation :

- Focus monitoring on Fair and Poor segments due to higher DTI.
- Maintain favorable strategies for higher credit score users.
- Investigate the Fair segment further for risk mitigation.

Average Credit Limit by Age

Syntax

```python
#aggregate card-level data to user level
#use 'min' for recency (most recent transaction)
#use 'sum' for monetary and transaction count columns
df_card_agg = df_cards.groupby(['client_id']).agg({
    'days_since_last_trx': 'min',
    'credit_limit': 'sum',
    'count_nonfraud_trx_L6M': 'sum',
    'amt_nonfraud_trx_L6M': 'sum',
    'count_fraud_trx_L6M': 'sum',
    'amt_fraud_trx_L6M': 'sum'
}).reset_index()

#show
df_card_agg.head(3)
```

	client_id	days_since_last_trx	credit_limit	count_nonfraud_trx_L6M	amt_nonfraud_trx_L6M	count_fraud_trx_L6M	amt_fraud_trx_L6M
0	0	3	165775000.0	685.0	535262100.0	0.0	0.0
1	1	20	65592000.0	498.0	264007900.0	0.0	0.0
2	10	604	108249000.0	0.0	0.0	0.0	0.0

```python
#merge aggregated card data with user data
#join on client_id (cards) and id (users)
df_user_agg = pd.merge(
    df_users,
    df_card_agg,
    how='left',
    left_on='id',
    right_on='client_id'
)

#show
df_user_agg.head(3)
```

	id	retirement_age	birthdate	gender	per_capita_income	yearly_income	total_debt	credit_score	age	retired_flag	DTI	credit_score_category	client_id	days_since_last_trx	credit_limit	count_nonfraud_trx_L6M	amt_nonfraud_trx_L6M	count_fraud_trx_L6M
0	825	66	1972-11-25	Female	45937000	93663000	38138095	787	52	0	0.407184	Very Good	825	10.0	164867000.0	599.0	755489800.0	1.0
1	1746	68	1972-12-16	Female	59451000	121212000	57186095	701	52	0	0.471786	Good	1746	24.0	102033000.0	189.0	388726600.0	0.0
2	1718	67	1944-11-04	Female	35586000	52535000	58666	698	80	1	0.001117	Good	1718	4.0	209210000.0	1322.0	705750600.0	0.0

Average Credit Limit by Age

Insight :

- Credit limit increases with age, reflecting financial maturity and income growth.
- Middle-aged users (40–70) show the most stable and higher credit limits.
- Older age groups show higher variability in credit limits.

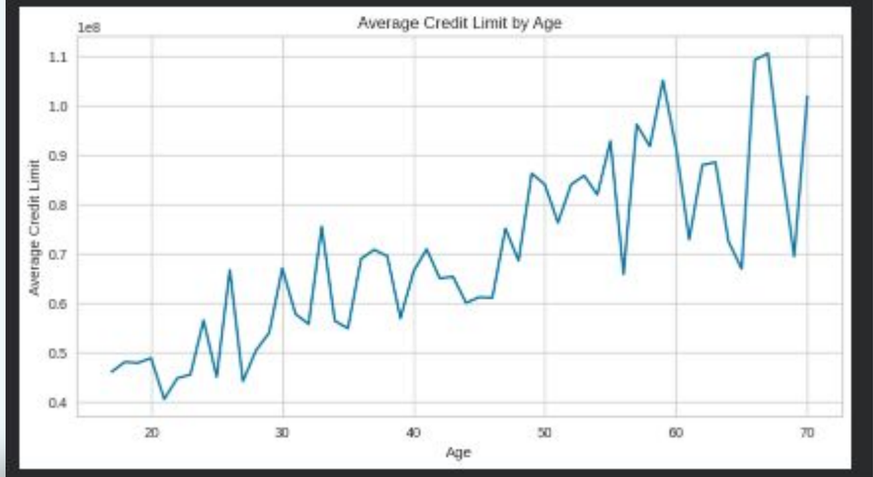
Recommendation :

- Focus on middle-aged users as the primary target segment for growth.
- Apply cautious credit limit increases for younger and older users.
- Use age as one of the factors for credit strategy, but not as the sole determinant.

```python
#cap age to reduce noise from extreme values
#ages above 70 are grouped into a single category (70)
df_user_agg['age_capped'] = np.where(df_user_agg['age'] > 70, 70, df_user_agg['age'])
```

```python
#calculate average credit limit per age group
#group by age and compute mean credit limit
credit_limit_per_age = df_user_agg.groupby(['age_capped']).agg({
    'credit_limit': 'mean'
}).reset_index()
```

```python
#visualize the relationship using a line chart
fig, ax = plt.subplots(figsize=(10,5))

ax.plot(
    credit_limit_per_age['age_capped'],
    credit_limit_per_age['credit_limit']
)

#add titles and labels
ax.set_title('Average Credit Limit by Age')
ax.set_xlabel('Age')
ax.set_ylabel('Average Credit Limit')

#display the chart
fig.show()
```



Segmentation Methodology

Clustering Method

K-Means Clustering was selected because it can group customers based on transaction behavior, financial capability, and engagement patterns using multiple numerical features.

Features Used

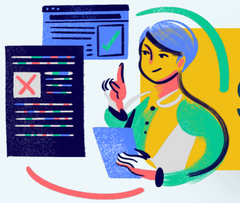
- Transaction Count
- Transaction Amount
- Days Since Last Transaction
- Income
- Credit Score
- Credit Limit
- DTI
- Age

Data Preparation

- Filtered active customers only
- Applied RobustScaler to handle outliers

Cluster Evaluation

- Elbow Method
- Silhouette Analysis
- Final selected cluster: k = 3



Segmentation Interpretation

Variable	Cluster 0	Cluster 1	Cluster 2
Avg Transaction Count	200 Trx	575 Trx	165 Trx
Avg Transaction Amount	182M	474M	104M
Avg Day SInce Last Transaction	14 Days	14 Days	17 Days
Avg Income	66M	41M	30M
Avg Credit Score	721	681	718
Avg Credit Limit	53M	26M	20M
Avg DTI	0.21	0.28	0.24
Avg Age	53	48	53

High Growth Potential

Very High Profit Potential

Low Engagement

Segmentation Recommendation

Based on the customer segmentation analysis, RevoBank should focus on increasing transaction activity and maximizing transaction-based revenue by applying different strategies for each customer segment.

1. Focus retention programs on **Cluster 1** (**High Value Active Customers**) because this segment generates the **highest transaction count and transaction amount**, making them the **largest contributor to RevoBank's MDR-based revenue**.
2. Increase spending engagement for **Cluster 0** (**Affluent Potential Customers**) through **premium lifestyle campaigns**, **installment promotions**, and **personalized offers**, since this segment has **high income and high credit limits** but has not fully optimized their spending capacity.
3. Improve customer activation for **Cluster 2** (**Low Engagement Customers**) by providing **cashback programs**, **transaction reminders**, and **beginner-friendly promotions** to encourage more frequent credit card usage.
4. Develop **personalized marketing strategies** based on customer behavior and financial profiles to improve **customer engagement**, **transaction frequency**, and **long-term profitability**.
5. Prioritize **transaction-focused campaigns** because RevoBank's revenue depends heavily on c**ustomer transaction activity through MDR fees**.

THANK YOU!!

By : Eva Arliana

Team 1 - Seoul - FSDA FEB26